

# Light-WAM: Efficient World Action Models with State-Fusion Action Decoding

Ziang Li<sup>1,2,\*</sup>, Dongzhou Cheng<sup>2,3,\*</sup>, Yibin Wang<sup>2,4</sup>, Shiyue Wang<sup>2,5</sup>,  
Xiaoyang Xu<sup>1</sup>, Lingxuan Weng<sup>5</sup>, Juan Wang<sup>1,†</sup>, Jiaqi Wang<sup>2,†</sup>

<sup>1</sup> Wuhan University    <sup>2</sup> Shanghai Innovation Institute

<sup>3</sup> Southeast University    <sup>4</sup> Fudan University    <sup>5</sup> East China Normal University

**Abstract:** World Action Models (WAMs) extend robot policy learning by incorporating future prediction as an additional training objective, encouraging the policy to encode task-relevant temporal structure in its representations. Current WAMs often rely on large-scale generative architectures that incur high training costs and inference latency, making them difficult to deploy as efficient closed-loop policies. We propose **Light-WAM**, a lightweight World Action Model for efficient robot manipulation. Specifically, it is built with a compact video backbone and performs future-video supervision in a downsampled latent space, reducing the cost of video co-training while retaining its benefits for representation learning. For action prediction, Light-WAM introduces the StateFusionAction-Expert, which reads adapted states from multiple backbone layers, fuses them through learned-query pooling, and directly predicts action chunks in a single forward pass. This design provides an efficient interface between video backbone representations and robot actions, avoiding the need for heavy generative action experts. Experiments demonstrate that Light-WAM maintains strong performance on LIBERO and achieves usable multi-task performance on RoboTwin 2.0, while using only 0.44B trainable parameters. It also achieves 72.03ms inference latency with 4.1GiB peak GPU memory and improved training throughput. The code is available at <https://github.com/L1ziang/Light-WAM>.

## 1 Introduction

Vision Language Action (VLA) models have shown strong performance in instruction-following robot manipulation by mapping visual observations and language instructions to robot actions [1, 2, 3, 4, 5, 6]. World Action Models (WAMs) extend this formulation by training robot policies jointly with future video prediction [7, 8, 9, 10, 11, 12]. The future-video objective provides additional supervision on how the scene changes over time, enabling the policy to learn representations that capture object motion, interaction dynamics, and task progress. However, current WAMs typically couple future-video prediction and action generation within large-scale generative architectures, resulting in substantial GPU memory usage, training cost, and inference latency. These overheads make it challenging to deploy WAMs as efficient closed-loop robot policies.

Recent work has shown that test-time future video generation is not necessary for strong policy performance, suggesting that the main benefit of video prediction may come from training-time representation learning [12]. Building on this, we study whether WAMs can be made more efficient while retaining the training benefit of future-video prediction. This leads to a compact WAM designed for efficient training and fast inference.

We propose **Light-WAM**, a lightweight World Action Model for efficient robot manipulation. Light-WAM uses Wan2.1-T2V-1.3B as the video backbone [13], keeps the pretrained backbone frozen, and adapts it with lightweight modules. To reduce the cost of video supervision, Light-WAM applies the

---

\*Equal contribution.

†Corresponding authors.

future-video objective in a downsampled latent space. During inference, Light-WAM predicts action chunks from the current observation, without test-time future-video generation or a generative action expert. To connect the video backbone to robot actions, we introduce the StateFusionActionExpert. This module reads adapted states from multiple backbone layers and compresses dense video tokens with learned-query pooling. The pooled states are fused and mapped to actions in a single forward pass. This provides an efficient interface between video representations and action prediction, while allowing the action decoder to use information from different levels of the video backbone.

We evaluate Light-WAM on LIBERO [14] and RoboTwin 2.0 [15]. On LIBERO, Light-WAM achieves 97.2% average success without embodied pretraining, which is competitive with larger WAM baselines. On RoboTwin 2.0, Light-WAM achieves 76.4% average success across 50 tasks. Compared with Fast-WAM [12], Light-WAM reduces trainable parameters from 6.02B to 0.44B, improves training throughput by  $4.25\times$ , and reduces inference latency to 72.03ms with 4.1GiB peak GPU memory. These results show that Light-WAM substantially improves the efficiency of the WAM pipeline while maintaining strong LIBERO performance and achieving usable multi-task performance in the more challenging RoboTwin 2.0.

Our contributions are summarized as follows:

- We propose Light-WAM, a lightweight World Action Model that combines a compact video backbone with downsampled latent-space video supervision, reducing the cost of WAM training while retaining the representation benefits of future-video co-training.
- We introduce the StateFusionActionExpert, a direct action decoder that bridges video backbone representations and robot actions. It fuses multi-level adapted states through learned-query pooling and predicts action chunks in a single forward pass.
- We evaluate Light-WAM on LIBERO and RoboTwin 2.0. Light-WAM achieves strong LIBERO performance and usable multi-task performance on RoboTwin 2.0, while substantially reducing both training and inference costs compared with heavier WAM baselines.

## 2 Related Work

**Vision Language Action Models.** Vision Language Action (VLA) models have become a central paradigm for instruction-following robot manipulation. Given visual observations and a language instruction, these models predict robot actions, enabling task conditioning and scalable learning from multi-task robot datasets [1, 2, 3, 5, 6, 16, 17, 18, 19, 20, 21]. Recent work further improves the practicality of VLA policies: SmolVLA [6] explores compact architectures for efficient training and deployment, while VLA-Adapter [21] introduces a lightweight interface for adapting vision-language representations to action prediction. However, these methods are primarily trained through action supervision, leaving the temporal structure of the task to be captured implicitly by the policy.

**World Action Models.** World Action Models (WAMs) provide a different perspective by coupling robot action learning with future video prediction. The future-video objective offers a temporal training signal that encourages the backbone to encode object motion, interaction dynamics, and task progress, leading to more world-aware visual representations [7, 8, 9, 10, 11, 12, 22, 23, 24, 25, 26, 27, 28]. Recent WAM systems such as Motus [10], LingBot-VA [11], and Fast-WAM [12] demonstrate the value of video co-training for robot policy learning, but they often rely on large video-action generative architectures and expensive training or inference pipelines.

Our work shares the efficiency-oriented goal of recent VLA policies, but targets WAM robot policies, where future video prediction is used to shape the visual representations for robot control. It is also closely related to Fast-WAM, which shows that the video prediction branch can be used as training-time supervision without being executed during inference. Rather than focusing on inference-time video rollout, we focus on improving the efficiency of the overall WAM pipeline.

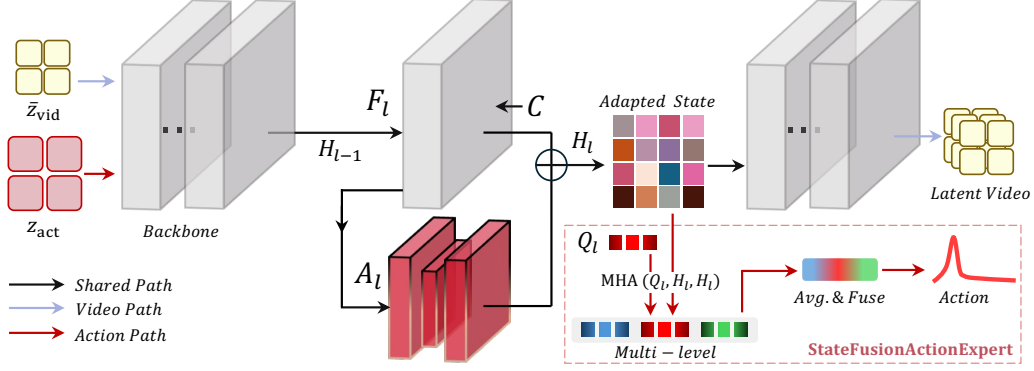


Figure 1: **Overview of Light-WAM.** Light-WAM shares an adapted video backbone between video co-training and action prediction. During training, the **video branch** applies future-video supervision to downsampled latent videos  $\bar{z}_{\text{vid}}$ , reducing the token cost of temporal supervision. The **action prediction branch** runs in both training and inference: it takes the current observation latent  $z_{\text{act}}$  and predicts action chunks without future-video rollout. The backbone is adapted with LoRA and sparse WAM adapters, and multi-level adapted states are fused by the StateFusionActionExpert through learned-query pooling for single-pass action decoding.

### 3 Methodology

#### 3.1 Overview

Building on this motivation, Light-WAM keeps future-video supervision during training, but instantiates the policy with a compact backbone and a direct action interface at test time. Given the current observation  $o$ , language instruction  $l$ , and proprioceptive state  $p$ , it predicts an action sequence by

$$\hat{A} = \pi_{\phi}(h_{\theta}(o, l, p)), \quad (1)$$

where  $h_{\theta}$  denotes the multi-level backbone representation extracted from the adapted video backbone, and  $\pi_{\phi}$  is the StateFusionActionExpert. Light-WAM is designed to make the WAM pipeline efficient in both training and inference. It uses a compact video backbone with minimal adaptation to preserve the pretrained video prior, reduces video supervision cost via latent-space downsampling, and employs the StateFusionActionExpert to decode actions directly from multi-level backbone states, enabling fast closed-loop execution without iterative action denoising. The overall architecture of Light-WAM is illustrated in Figure 1. Detailed procedures are provided in Appendix A.

#### 3.2 Video Backbone Adaptation

Light-WAM uses Wan2.1-T2V-1.3B as the video backbone [13]. Given a VAE latent input  $z$ , the patch embedding layer produces the initial video-token state:

$$H_0 = \text{PatchEmbed}(z) \in \mathbb{R}^{B \times N \times d}, \quad (2)$$

where  $N$  is the number of spatiotemporal video tokens and  $d$  is the hidden dimension. The language instruction is encoded as text context tokens, and the proprioceptive state is projected to the same context dimension and appended to them:

$$C = [c_1, \dots, c_L, c_{\text{prop}}]. \quad (3)$$

The backbone then updates the video-token state through transformer blocks, with  $C$  provided as the cross-attention context. To preserve the pretrained video prior, we freeze the original Wan backbone and adapt it through low-rank updates on its attention and feed-forward projections [29]. We further insert lightweight WAM adapters at a sparse set of backbone depths. Let  $F_{\ell}$  denote the  $\ell$ -th transformer block and  $A_{\ell}$  the WAM adapter inserted at that depth, if present. Given the previous

hidden state  $H_{\ell-1}$  and context  $C$ , the layer update is

$$U_\ell = F_\ell(H_{\ell-1}, C), \quad H_\ell = \begin{cases} U_\ell + A_\ell(U_\ell), & \ell \in \mathcal{I}, \\ U_\ell, & \text{otherwise,} \end{cases} \quad (4)$$

where  $\mathcal{I}$  denotes the depths exposed to the action decoder and  $A_\ell$  is a lightweight bottleneck MLP. In this way, low-rank updates provide lightweight adaptation across the backbone, while the sparse WAM adapters provide additional robot-domain adaptation capacity at selected depths. The action branch reads a sparse set of adapted backbone states:

$$\mathcal{H} = \{H_\ell\}_{\ell \in \mathcal{I}}. \quad (5)$$

These multi-level backbone states form the interface between the video backbone and the StateFusionActionExpert. Instead of using only the final representation or exposing all backbone activations, Light-WAM selects a small set of states from different backbone levels, allowing the action head to access visual information at multiple granularities while keeping action decoding efficient.

### 3.3 Efficient Latent Video Co-training

The future-video branch provides temporal supervision during training. Let  $G_\theta^{\text{vid}}$  denote the video prediction branch, which includes the adapted video backbone and the final video prediction head. Let  $\bar{z}_{\text{vid}} = D(z_{\text{vid}})$  be the latent video after spatial downsampling, and let  $\bar{z}_t$  denote its flow-matching perturbation at time  $t$ . The video branch is optimized by

$$\mathcal{L}_{\text{video}} = \|G_\theta^{\text{vid}}(\bar{z}_t, t, C) - u_t\|_2^2, \quad (6)$$

where  $u_t$  is the corresponding flow-matching target [30]. The first latent frame is also downsampled by  $D(\cdot)$  and kept fixed in  $\bar{z}_t$  as the observation condition. For action prediction, Light-WAM takes the current observation from the original-resolution latent video:

$$z_{\text{act}} = z_{\text{vid}}^{(0)}, \quad (7)$$

and does not apply the additional spatial downsampling used for video supervision. Thus, the video branch learns future dynamics in a lower-cost downsampled latent space, while the action branch preserves the original-resolution current observation needed for manipulation.

### 3.4 Query-Bottlenecked State Fusion and Action Decoding

Given the full-resolution current observation latent  $z_{\text{act}}$ , Light-WAM runs the adapted video backbone once and obtains the multi-level backbone states  $\mathcal{H} = \{H_\ell\}_{\ell \in \mathcal{I}}$ . The StateFusionActionExpert converts these dense video-token states into a fixed-width action state through learned-query pooling. This design is related to prior query-based pooling methods that use learnable queries to compress dense input tokens into compact representations [31, 32]. For each backbone state  $H_\ell \in \mathcal{H}$ , we learn a set of query tokens

$$Q_\ell \in \mathbb{R}^{N_q \times d}. \quad (8)$$

The query tokens attend to the video tokens of the corresponding backbone level, producing  $P_\ell \in \mathbb{R}^{B \times N_q \times d}$ , which is then averaged over queries and normalized:

$$P_\ell = \text{MHA}(Q_\ell, H_\ell, H_\ell), \quad s_\ell = \text{LN} \left( \frac{1}{N_q} \sum_{j=1}^{N_q} P_{\ell,j} \right). \quad (9)$$

MHA and LN denote multi-head attention [33] and layer normalization [34], respectively. The number of queries controls the information passed from the video backbone to the action head. Too few queries may lose manipulation-relevant visual details, while too many reduce the compression effect and increase the burden on the action decoder. This design provides a controlled bottleneck that compresses dense video tokens into level-wise state representations. The resulting states are projected and fused as

$$h = \phi_{\text{trunk}}(\phi_{\text{fuse}}([M_\ell(s_\ell)]_{\ell \in \mathcal{I}})). \quad (10)$$

To decode actions, Light-WAM uses step embeddings  $\{e_k\}_{k=1}^K$ , where  $K$  is the action horizon. Each embedding is projected by  $\psi(\cdot)$  and added to the fused state  $h$ , after which an output head predicts the corresponding action:

$$r_k = h + \psi(e_k), \quad \hat{a}_k = \phi_{\text{out}}(\text{LN}(r_k)), \quad \hat{A} = [\hat{a}_1, \dots, \hat{a}_K] \in \mathbb{R}^{B \times K \times d_a}. \quad (11)$$

The full training objective combines future-video supervision and action regression:

$$\mathcal{L} = \mathcal{L}_{\text{video}} + \lambda \mathcal{L}_{\text{action}}(\hat{A}, A), \quad (12)$$

where  $A$  denotes the target action sequence and  $\mathcal{L}_{\text{action}}$  measures the regression error. At inference time, Light-WAM directly predicts actions from the current observation without future-video rollout.

## 4 Experiments

### 4.1 Experimental Setup

**Benchmarks and data.** We evaluate Light-WAM on LIBERO [14] and RoboTwin 2.0 [15]. For LIBERO, we use the official datasets and report success rates on four suites: Spatial, Object, Goal, and Long. For RoboTwin 2.0, we follow the multi-task evaluation protocol used in prior work [10, 11, 12]: one policy is trained on 50 tasks using 2,500 clean demonstrations and 25,000 randomized demonstrations. We report performance under both clean and randomized evaluation settings.

**Implementation details.** Light-WAM uses Wan2.1-T2V-1.3B [13] as a frozen video backbone and trains only the lightweight adaptation and action prediction modules. We insert WAM adapters at layers  $\{8, 16, 24\}$ , set the number of learned queries to 16 for each selected layer, and apply  $2\times$  spatial latent downsampling in the video co-training branch. The default model has 1.99B total parameters and 0.44B trainable parameters. We train with AdamW using learning rate  $1e-4$ , weight decay  $1e-2$ , LIBERO batch size 64, and RoboTwin 2.0 batch size 128. Training is conducted on 4 NVIDIA H100 GPUs, and inference is measured on NVIDIA RTX 4090 48G GPUs. Additional implementation details are provided in Appendix B.

**Baselines and metrics.** We compare Light-WAM with representative VLA and WAM policies, including OpenVLA [3], OpenVLA-OFT [19], VLA-Adapter [21],  $\pi_0$  [5],  $\pi_{0.5}$  [18], X-VLA [35], Motus [10], LingBot-VA [11], and Fast-WAM [12]. Since large-scale embodied pretraining can significantly affect downstream manipulation performance, we indicate whether each method uses it. For task performance, we report success rate. For efficiency, we report trainable parameters, training throughput, inference latency, and peak GPU memory.

### 4.2 LIBERO Results

Table 1 reports LIBERO results. Light-WAM achieves 97.2% average success, ranking first among methods without embodied pretraining and third among all compared methods. This indicates that Light-WAM remains competitive on LIBERO with fewer parameters than existing WAM baselines.

Light-WAM obtains 98.2%, 99.6%, 97.8%, and 93.0% on Spatial, Object, Goal, and Long, respectively. The Long suite remains the most challenging setting, where larger policies such as Motus and LingBot-VA achieve higher success rates, suggesting that long-horizon tasks can still benefit from larger model capacity. Overall, the LIBERO results show that Light-WAM achieves competitive task performance with improved model efficiency.

### 4.3 Multi-Task Learning on RoboTwin 2.0

We further evaluate Light-WAM on RoboTwin 2.0 to study whether the lightweight architecture remains usable in a larger multi-task setting. Unlike LIBERO, RoboTwin 2.0 requires a single policy to learn across 50 bimanual manipulation tasks and handle randomized visual and physical

Table 1: **LIBERO success rates on the four official suites.** We report average success, rank among methods without embodied pretraining (w/o EPT), and overall rank.

| Type | Method           | Params | EPT | Spatial | Object | Goal | Long | Avg. | w/o EPT Rank | Overall Rank |
|------|------------------|--------|-----|---------|--------|------|------|------|--------------|--------------|
| VLA  | OpenVLA [3]      | 7B     | w/  | 84.7    | 88.4   | 79.2 | 53.7 | 76.5 | —            | 9            |
|      | OpenVLA-OFT [19] | 7B     | w/  | 97.6    | 98.4   | 97.9 | 94.5 | 97.1 | —            | 4            |
|      | VLA-Adapter [21] | 0.6B   | w/o | 96.0    | 96.8   | 97.4 | 94.4 | 96.2 | 3            | 7            |
|      | $\pi_0$ [5]      | 3B     | w/  | 96.8    | 98.8   | 95.8 | 85.2 | 94.1 | —            | 8            |
|      | $\pi_{0.5}$ [18] | 3B     | w/  | 98.8    | 98.2   | 98.0 | 92.4 | 96.9 | —            | 6            |
| WAM  | Motus [10]       | 8B     | w/  | 96.8    | 99.8   | 96.6 | 97.6 | 97.7 | —            | 2            |
|      | LingBot-VA [11]  | 5.3B   | w/  | 98.5    | 99.6   | 97.2 | 98.5 | 98.5 | —            | 1            |
|      | Fast-WAM [12]    | 6B     | w/o | 97.0    | 99.4   | 96.6 | 94.8 | 97.0 | 2            | 5            |
|      | <b>Light-WAM</b> | 2B     | w/o | 98.2    | 99.6   | 97.8 | 93.0 | 97.2 | <b>1</b>     | 3            |

Table 2: **RoboTwin 2.0 success rates on 50 tasks.** We report clean, randomized, and average success.

| Type | Method           | Params | EPT | Clean | Randomized | Avg. |
|------|------------------|--------|-----|-------|------------|------|
| VLA  | $\pi_0$ [5]      | 3B     | w/  | 65.9  | 58.4       | 62.2 |
|      | $\pi_{0.5}$ [18] | 3B     | w/  | 82.7  | 76.8       | 79.8 |
|      | X-VLA [35]       | 0.9B   | w/  | 72.9  | 72.8       | 72.9 |
| WAM  | Motus [10]       | 8B     | w/  | 88.7  | 87.0       | 87.8 |
|      | Motus [10]       | 8B     | w/o | 72.8  | 77.0       | 74.9 |
|      | LingBot-VA [11]  | 5.3B   | w/  | 92.9  | 91.5       | 92.2 |
|      | LingBot-VA [11]  | 5.3B   | w/o | 80.6  | —          | 80.6 |
|      | Fast-WAM [12]    | 6B     | w/o | 91.9  | 91.8       | 91.9 |
|      | <b>Light-WAM</b> | 2B     | w/o | 76.4  | 76.3       | 76.4 |

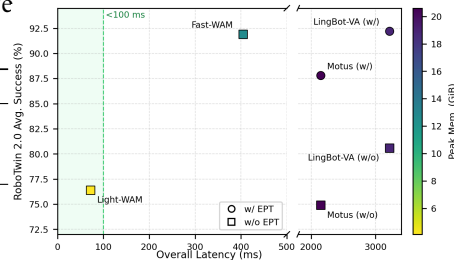


Figure 2: RoboTwin 2.0 inference efficiency-performance comparison.

conditions. This setting is more challenging for a lightweight model such as Light-WAM, with only 0.44B trainable parameters and a direct action head rather than large generative action experts.

As shown in Table 2, Light-WAM achieves 76.4% average success on RoboTwin 2.0 without embodied pretraining. Although it does not match Fast-WAM or the strongest embodied-pretrained WAMs, this result shows that Light-WAM can obtain usable multi-task performance with a much smaller trainable parameter budget. It outperforms  $\pi_0$  and X-VLA in this comparison, and is competitive with Motus without embodied pretraining. These results position Light-WAM as an efficient WAM policy. While larger models perform better in the more complex RoboTwin 2.0 setting, Light-WAM achieves usable multi-task performance with much lower training and inference cost. Figure 2 visualizes the inference-side trade-off: Light-WAM achieves much lower inference latency and peak GPU memory among WAM methods, while maintaining usable average success on RoboTwin 2.0.

#### 4.4 Efficiency Analysis

Light-WAM is designed to reduce the cost of the entire WAM training and inference pipeline. Table 3 reports training efficiency. Compared with Fast-WAM, Light-WAM reduces total training-time parameters from 6.73B to 1.99B and trainable parameters from 6.02B to 0.44B, corresponding to  $3.4\times$  and  $13.7\times$  reductions, respectively. Peak per-GPU memory decreases from 70.7GiB to 43.1GiB, and throughput increases from 0.49 to 2.08 steps/s.

We further analyze the contribution of each efficiency component. A compact video backbone alone does not guarantee faster training, partly because the Wan2.1 VAE produces a denser latent grid than the high-compression VAE used by Wan2.2-TI2V-5B [13]. Introducing the StateFusionActionExpert reduces the action-side parameter and computation cost. Latent caching removes online VAE encoding from the training loop, and  $2\times$  spatial downsampling reduces the token cost of future-video co-training. Together, these choices reduce training cost while preserving future-video supervision as part of the learning objective.

Table 3: **Training efficiency analysis.** We measure on  $4\times$  NVIDIA H100 GPUs with effective global batch size 64. Loaded Params denotes training-time loaded model parameters.

| Model / Variant  | Action Head | Latent Cache | Video Downsample | Loaded Params | Trainable Params | Mem. / GPU | Samples/s | Steps/s | Steps/s Norm.                  |
|------------------|-------------|--------------|------------------|---------------|------------------|------------|-----------|---------|--------------------------------|
| Fast-WAM [12]    | DiT [36]    | No           | $1\times$        | 6.73B         | 6.02B            | 70.7GiB    | 31.6      | 0.49    | $1.00\times$                   |
| Light-WAM        | DiT [36]    | No           | $1\times$        | 2.28B         | 0.73B            | 58.9GiB    | 27.7      | 0.43    | $0.88\times$                   |
| Light-WAM*       | StateFusion | No           | $1\times$        | 1.99B         | 0.44B            | 48.6GiB    | 35.7      | 0.56    | $1.14\times$                   |
| Light-WAM*       | StateFusion | Yes          | $1\times$        | 1.99B         | 0.44B            | 48.2GiB    | 55.3      | 0.86    | $1.76\times$                   |
| <b>Light-WAM</b> | StateFusion | Yes          | $2\times$        | 1.99B         | 0.44B            | 43.1GiB    | 133.3     | 2.08    | <b><math>4.25\times</math></b> |

\* These variants use batch size 8 per GPU and gradient accumulation 2 to avoid OOM.

Table 4: **Inference efficiency on RoboTwin 2.0 inputs.** Latency is measured per action query with cached language context on a single NVIDIA RTX 4090 48GB GPU. Overall latency includes VAE encoding and policy forward, while simulator and I/O overheads are excluded.

| Model            | Params | Prediction Scope | VAE Enc. | Visual Branch | Action Branch | Policy Forward | Peak Mem.     | Overall Latency | Norm.                          |
|------------------|--------|------------------|----------|---------------|---------------|----------------|---------------|-----------------|--------------------------------|
| $\pi_{0.5}$ [18] | 3B     | Action-only      | —        | —             | —             | —              | > 8GiB        | 76ms*           | $1.00\times$                   |
| LingBot-VA [11]  | 5.3B   | Video + action   | 223.8ms  | —             | —             | 2990.1ms       | 18.9GiB       | 3214.14ms       | $42.29\times$                  |
| Motus [10]       | 8B     | Video + action   | 16.1ms   | —             | —             | 2130.7ms       | 20.6GiB       | 2148.68ms       | $28.27\times$                  |
| Fast-WAM [12]    | 6B     | Action-only      | 11.3ms   | 36.0ms        | 356.8ms       | 392.8ms        | 12.7GiB       | 404.62ms        | $5.32\times$                   |
| <b>Light-WAM</b> | 2B     | Action-only      | 12.7ms   | 56.5ms        | 2.1ms         | 58.6ms         | <b>4.1GiB</b> | <b>72.03ms</b>  | <b><math>0.95\times</math></b> |

\* The  $\pi_{0.5}$  latency is reported by [37], which also evaluates on an NVIDIA RTX 4090 GPU.

Table 4 reports inference efficiency on RoboTwin 2.0 inputs. Latency is measured per action query with cached language context, including VAE encoding and policy forward. Light-WAM achieves 72.03ms overall latency with 4.1GiB peak GPU memory, substantially lower than prior WAM methods. The breakdown shows that its action branch takes only 2.1ms, while larger WAMs spend much more time on iterative action prediction or joint video-action generation. These results show that Light-WAM enables fast and memory-efficient action prediction for closed-loop control.

#### 4.5 Ablation Studies

We conduct ablations on LIBERO-Spatial for three Light-WAM designs: the resolution of video co-training, the number of adapter layers, and the capacity of learned-query pooling. As shown in Table 5, using the original-resolution video latent for co-training improves success from 98.2% to 99.0%. This suggests that higher-resolution video supervision can further improve policy performance. However, full-resolution video co-training raises the training cost substantially, as shown in Table 3. We therefore use  $2\times$  latent downsampling to balance performance and training efficiency.

Increasing the number of adapter layers from 3 to 5 gives similar performance, with success changing from 98.2% to 98.0%. This indicates that adding more adapter layers brings no clear gain in this setting. Considering the additional parameters and computation, we choose a sparse three-layer configuration  $\{8, 16, 24\}$ , which provides multi-level representations for the action decoder. Finally, reducing the number of learned queries from 16 to 8 decreases success to 95.4%, suggesting that the query bottleneck needs sufficient capacity to preserve manipulation-relevant visual information.

Table 5: **Ablations on LIBERO-Spatial.** The default Light-WAM uses  $2\times$  latent downsampling, adapters at layers  $\{8, 16, 24\}$ , and 16 learned queries.

| Variant              | DS        | Adapter Layers         | Queries | Success |
|----------------------|-----------|------------------------|---------|---------|
| <b>Light-WAM</b>     | $2\times$ | $\{8, 16, 24\}$        | 16      | 98.2    |
| w/o downsample       | $1\times$ | $\{8, 16, 24\}$        | 16      | 99.0    |
| w/ 5 adapter layers  | $2\times$ | $\{4, 8, 16, 20, 24\}$ | 16      | 98.0    |
| w/ 8 learned queries | $2\times$ | $\{8, 16, 24\}$        | 8       | 95.4    |

**Overall,** Light-WAM achieves competitive LIBERO performance, usable 50-task performance on RoboTwin 2.0, and much lower training and inference cost. These results support the main design of Light-WAM: future-video prediction is retained as downsampled latent-space supervision, while multi-level adapted backbone states are fused by a single-pass StateFusionActionExpert for action decoding.

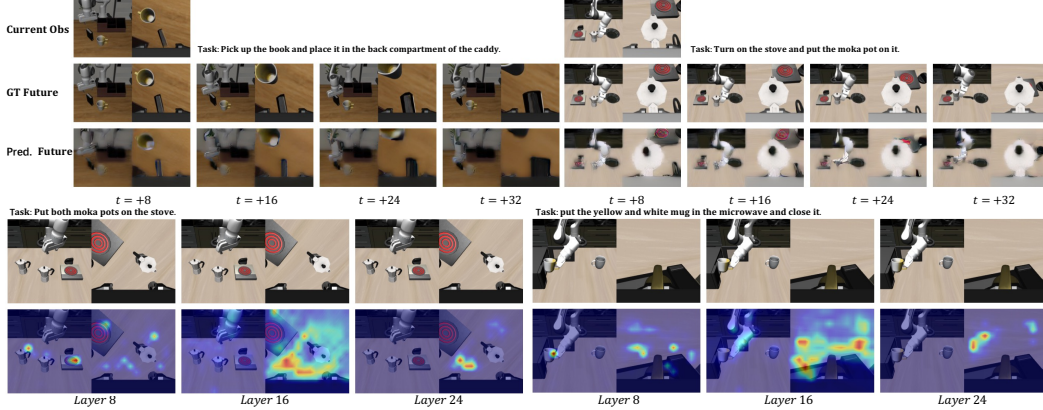


Figure 3: **Qualitative analysis.** Top: future-video predictions compared with reference rollout frames at  $t = \{+8, +16, +24, +32\}$ . Bottom: learned-query visualizations from the StateFusionActionExpert.

#### 4.6 Qualitative Analysis

**Future video visualization.** The top row of Figure 3 shows examples from the video branch. For each task, we compare the predicted future frames with reference future frames from the environment rollout. The predictions are smoother than the reference frames because the video branch is trained in a downsampled latent space. However, they still capture the main motion and scene changes, suggesting that the video branch learns useful temporal information during training.

**Learned-query visualization.** The bottom row of Figure 3 visualizes attention maps derived from learned-query pooling. When projected back to image space, the maps from layers 8, 16, and 24 tend to emphasize different task-relevant regions, such as manipulated objects, the gripper, and target areas. This suggests that the selected backbone layers provide complementary visual cues, which is consistent with our design of fusing multi-level adapted states for action decoding.

#### 4.7 Real-World Evaluation

We evaluate Light-WAM on the IMETA Y1 dual-arm robot platform with three real-world manipulation tasks. For each task, we collect 50 demonstrations for training and compare Light-WAM with  $\pi_{0.5}$  under the same setting. Figure 4 shows the robot setup, task observations, and success rates for the three tasks. Additional rollout frames are provided in Appendix D.

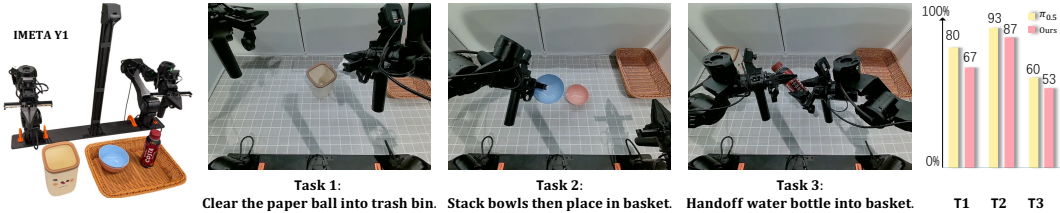


Figure 4: **Real-world evaluation.** Robot setup and success rates on three dual-arm tasks.

## 5 Conclusion and Limitations

We presented Light-WAM, a lightweight World Action Model for efficient robot manipulation. By combining a compact video backbone, downsampled latent-space video supervision, and the StateFusionActionExpert, Light-WAM improves the efficiency of both WAM training and inference. Experiments on LIBERO, RoboTwin 2.0, and real-world dual-arm tasks show a favorable performance-efficiency trade-off. There are also several limitations. In more challenging multi-task settings, larger WAMs and embodied-pretrained policies continue to achieve higher success rates, suggesting that model capacity and large-scale embodied data remain important for complex manipulation. Moreover, although we evaluate on existing benchmarks and real-world tasks, **we do not train or test on benchmarks specifically designed for policy generalization and robustness**, such as LIBERO-Plus [38]. Future work will incorporate data augmentation and robustness-oriented training to further improve the generalization ability of Light-WAM.

## References

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [4] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- [5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al.  $\pi_0$ : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [6] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [7] J. Liang, P. Tokmakov, R. Liu, S. Sudhakar, P. Shah, R. Ambrus, and C. Vondrick. Video generators are robot policies. *arXiv preprint arXiv:2508.00795*, 2025.
- [8] S. Li, Y. Gao, D. Sadigh, and S. Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- [9] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.
- [10] H. Bi, H. Tan, S. Xie, Z. Wang, S. Huang, H. Liu, R. Zhao, Y. Feng, C. Xiang, Y. Rong, et al. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025.
- [11] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [12] T. Yuan, Z. Dong, Y. Liu, and H. Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [13] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [14] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [15] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [16] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

- [17] G. R. Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [18] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al.  $\pi_{0.5}$ : a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [19] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [20] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, volume 2025, pages 29982–30009, 2025.
- [21] Y. Wang, P. Ding, L. Li, C. Cui, Z. Ge, X. Tong, W. Song, H. Zhao, W. Zhao, P. Hou, et al. Vlada: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 18638–18646, 2026.
- [22] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.
- [23] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, volume 2024, pages 10641–10662, 2024.
- [24] H. Bharadhwaj, D. Dwibedi, A. Gupta, S. Tulsiani, C. Doersch, T. Xiao, D. Shah, F. Xia, D. Sadigh, and S. Kirmani. Gen2act: Human video generation in novel scenarios enables generalizable robot manipulation. *arXiv preprint arXiv:2409.16283*, 2024.
- [25] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan. Robodreamer: Learning compositional world models for robot imagination. *arXiv preprint arXiv:2404.12377*, 2024.
- [26] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.
- [27] Y. Liao, P. Zhou, S. Huang, D. Yang, S. Chen, Y. Jiang, Y. Hu, J. Cai, S. Liu, J. Luo, et al. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025.
- [28] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [29] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [30] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [31] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pages 3744–3753. PMLR, 2019.
- [32] J. Li, D. Li, S. Savarese, and S. Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.

- [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [34] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [35] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [36] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
- [37] K. Black, M. Galliker, and S. Levine. Real-time execution of action chunking flow policies. *Advances in Neural Information Processing Systems*, 38:33383–33407, 2026.
- [38] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

## A Algorithmic Details

**Backbone adaptation.** Light-WAM uses Wan2.1-T2V-1.3B as the video backbone and keeps the pretrained backbone weights frozen. We adapt the backbone with two lightweight components. First, LoRA is applied to the self-attention, cross-attention, and feed-forward projections of all backbone blocks. Second, sparse WAM adapters are inserted at layers  $\{8, 16, 24\}$ . Each WAM adapter is a residual bottleneck module:

$$A_\ell(x) = \gamma W_\ell^{\text{up}} \sigma(W_\ell^{\text{down}} x),$$

where  $W_\ell^{\text{down}}$  maps the backbone hidden state to a 256-dimensional bottleneck,  $W_\ell^{\text{up}}$  maps it back to the backbone hidden dimension, and  $\gamma$  is the adapter scale. For a selected layer  $\ell$ , the adapted state is computed as

$$H_\ell = U_\ell + A_\ell(U_\ell),$$

where  $U_\ell$  denotes the output of the corresponding backbone block. In our default configuration,  $\gamma = 1.0$ . The adapted states from the selected layers are exposed to the StateFusionActionExpert for action prediction, while the final backbone output is used by the video prediction head for future-video co-training.

**State-fusion.** The StateFusionActionExpert maps the selected adapted backbone states to action chunks through query-based pooling and lightweight state fusion. For each selected layer  $\ell \in \mathcal{I}$ , we use a layer-specific set of learnable queries  $Q_\ell$  to attend to the adapted video tokens  $H_\ell$ :

$$P_\ell = \text{MHA}(Q_\ell, H_\ell, H_\ell), \quad s_\ell = \text{LN} \left( \frac{1}{N_q} \sum_{j=1}^{N_q} P_{\ell,j} \right).$$

In our default configuration, each layer uses  $N_q = 16$  queries and 8 attention heads. The pooled state  $s_\ell$  is projected to a 4608-dimensional feature, and the features from layers  $\{8, 16, 24\}$  are concatenated and projected to a 6144-dimensional fused state. A single residual MLP block further processes the fused state. For temporal decoding, sinusoidal step-position embeddings of width 256 are projected and added to the fused state, after which an output MLP predicts the action at each step. For RoboTwin 2.0, the decoder outputs a  $24 \times 14$  action chunk.

## B Training and Implementation Details

**Training setup.** We train Light-WAM with AdamW, using a learning rate of  $1 \times 10^{-4}$ , weight decay of  $1 \times 10^{-2}$ , and a cosine learning-rate schedule with 1,000 warmup steps. All models are trained on 4 NVIDIA H100 GPUs. For LIBERO, we use a global batch size of 64. For RoboTwin 2.0, we use a global batch size of 128. Training uses cached Wan2.1 VAE latents to remove online VAE encoding from the training loop, while evaluation uses online VAE encoding. The video backbone weights are frozen, and the trainable components include the backbone LoRA modules, WAM adapters, video prediction head, proprio encoder, and StateFusionActionExpert.

**Checkpoint selection.** For LIBERO, we select checkpoints for each suite: 60K steps for Spatial and Goal, 12.5K steps for Object, and 80K steps for Long. For RoboTwin 2.0, we evaluate the model trained for 460K steps.

**Parameter breakdown.** Table 6 reports the parameter composition of the default Light-WAM model. The model has 1.99B total parameters, of which 0.44B are trainable. Most trainable parameters come from the StateFusionActionExpert and backbone LoRA modules, while the pretrained video backbone and VAE remain frozen.

---

**Algorithm 1** Light-WAM training on RoboTwin 2.0

---

**Require:** Observation sequence  $I_{0:32}$ , target action chunk  $A = \{a_k\}_{k=0}^{K-1}$ , language embedding  $c$ , proprioceptive states  $p_{0:K-1}$

**Require:** Selected adapter layers  $\mathcal{I} = \{8, 16, 24\}$

- 1: Construct the RoboTwin canvas video  $V$  from the three camera streams, and subsample frames with stride 4:

$$V_{\text{sub}} = [I_0, I_4, I_8, \dots, I_{32}].$$

- 2: Encode  $V_{\text{sub}}$  into Wan2.1 VAE latents  $z_{\text{vid}}$  using cached latents when available.
- 3: Build the cross-attention context

$$C = [c_1, \dots, c_{128}, c_{\text{prop}}].$$

▷ Future-video co-training branch

- 4: Spatially downsample the video latents,  $\bar{z}_{\text{vid}} = D(z_{\text{vid}})$ , and sample a flow-matching timestep  $t$  and noise  $\epsilon$ .
- 5: Construct the perturbed latent  $\bar{z}_t$  from  $\bar{z}_{\text{vid}}$ , while keeping the first latent frame fixed as the observation anchor.
- 6: Predict the flow target with the adapted video backbone and video head:

$$\hat{u}_t = G_{\theta}^{\text{vid}}(\bar{z}_t, t, C), \quad \mathcal{L}_{\text{video}} = \|\hat{u}_t - u_t\|_2^2.$$

▷ Action prediction branch

- 7: Take the current observation latent at the original latent resolution:

$$z_{\text{act}} = z_{\text{vid}}^{(0)}.$$

- 8: Run the adapted video backbone on  $z_{\text{act}}$  and collect multi-level adapted states:

$$\mathcal{H} = \{H_{\ell}\}_{\ell \in \mathcal{I}} = h_{\theta}(z_{\text{act}}, C).$$

- 9: Predict the action chunk with the StateFusionActionExpert:

$$\hat{A} = \{\hat{a}_k\}_{k=0}^{K-1} = \pi_{\phi}(\mathcal{H}).$$

- 10: Compute the weighted action regression loss:

$$\mathcal{L}_{\text{action}} = \sum_{k=0}^{K-1} w_k \|\hat{a}_k - a_k\|_2^2.$$

▷ Joint optimization

- 11: Update the trainable parameters using

$$\mathcal{L} = \mathcal{L}_{\text{video}} + \mathcal{L}_{\text{action}}.$$

---

**Algorithm 2** Light-WAM inference

---

**Require:** Current observation  $I_t$ , language embedding  $c$ , proprioceptive state  $p_t$

**Require:** Selected adapter layers  $\mathcal{I} = \{8, 16, 24\}$

- 1: Build the current observation image from the camera inputs and encode it into a single-frame latent  $z_{\text{act}}$ .
- 2: Build the cross-attention context

$$C = [c_1, \dots, c_{128}, c_{\text{prop}}],$$

where  $c_{\text{prop}}$  is obtained by projecting  $p_t$ .

- 3: Run one adapted video-backbone forward pass and collect selected adapted states:

$$\mathcal{H} = \{H_{\ell}\}_{\ell \in \mathcal{I}} = h_{\theta}(z_{\text{act}}, C).$$

- 4: Predict the action chunk:

$$\hat{A} = \{\hat{a}_k\}_{k=0}^{K-1} = \pi_{\phi}(\mathcal{H}).$$

- 5: Execute the predicted actions.
-

Table 6: Parameter breakdown of Light-WAM. Numbers are reported in millions of parameters.

| Component               | Total           | Trainable      | Frozen          |
|-------------------------|-----------------|----------------|-----------------|
| Frozen video backbone   | 1418.90M        | 0.00M          | 1418.90M        |
| Backbone LoRA modules   | 87.49M          | 87.49M         | 0.00M           |
| WAM adapters            | 2.37M           | 2.37M          | 0.00M           |
| Video prediction head   | 0.10M           | 0.10M          | 0.00M           |
| StateFusionActionExpert | 351.03M         | 351.03M        | 0.00M           |
| Proprio encoder         | 0.04M           | 0.04M          | 0.00M           |
| Wan VAE                 | 126.89M         | 0.00M          | 126.89M         |
| <b>Total</b>            | <b>1986.82M</b> | <b>441.03M</b> | <b>1545.79M</b> |

## C Full RoboTwin 2.0 Results

Table 7: Full RoboTwin 2.0 per-task results.

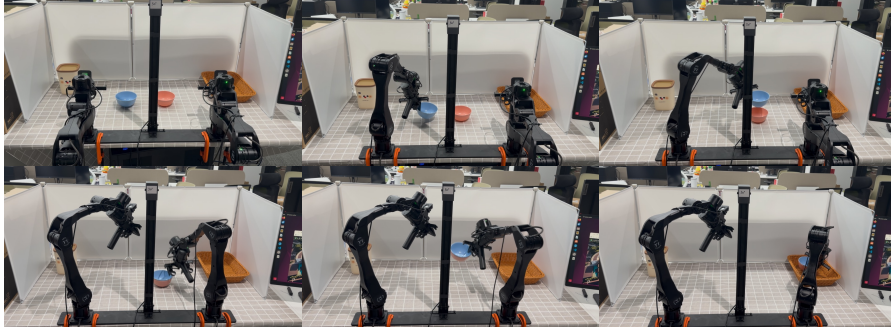
| Task                      | $\pi_{0.5}$ |             | X-VLA       |             | Fast-WAM    |             | Light-WAM   |             |
|---------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                           | Clean       | Rand.       | Clean       | Rand.       | Clean       | Rand.       | Clean       | Rand.       |
| Adjust Bottle             | 100         | 99          | 100         | 99          | 100         | 100         | 100         | 100         |
| Beat Block Hammer         | 96          | 93          | 92          | 88          | 99          | 97          | 83          | 80          |
| Blocks Ranking RGB        | 92          | 85          | 83          | 83          | 100         | 100         | 96          | 91          |
| Blocks Ranking Size       | 49          | 26          | 67          | 74          | 94          | 98          | 57          | 54          |
| Click Alarmclock          | 98          | 89          | 99          | 99          | 100         | 100         | 100         | 100         |
| Click Bell                | 99          | 66          | 100         | 100         | 100         | 100         | 100         | 100         |
| Dump Bin Bigbin           | 92          | 97          | 79          | 77          | 97          | 96          | 81          | 75          |
| Grab Roller               | 100         | 100         | 100         | 100         | 100         | 100         | 100         | 98          |
| Handover Block            | 66          | 57          | 73          | 37          | 95          | 81          | 71          | 59          |
| Handover Mic              | 98          | 97          | 0           | 0           | 99          | 100         | 90          | 94          |
| Hanging Mug               | 18          | 17          | 23          | 27          | 58          | 62          | 25          | 17          |
| Lift Pot                  | 96          | 85          | 99          | 100         | 100         | 100         | 93          | 93          |
| Move Can Pot              | 51          | 55          | 89          | 86          | 90          | 88          | 57          | 74          |
| Move Pillbottle Pad       | 84          | 61          | 73          | 71          | 100         | 99          | 69          | 74          |
| Move Playingcard Away     | 96          | 84          | 93          | 98          | 100         | 100         | 93          | 92          |
| Move Stapler Pad          | 56          | 42          | 78          | 73          | 77          | 64          | 26          | 34          |
| Open Laptop               | 90          | 96          | 93          | 100         | 98          | 100         | 91          | 97          |
| Open Microwave            | 34          | 77          | 79          | 71          | 62          | 45          | 76          | 59          |
| Pick Diverse Bottles      | 81          | 71          | 58          | 36          | 80          | 85          | 61          | 57          |
| Pick Dual Bottles         | 93          | 63          | 47          | 36          | 100         | 96          | 90          | 63          |
| Place A2B Left            | 87          | 82          | 48          | 49          | 95          | 93          | 84          | 83          |
| Place A2B Right           | 87          | 84          | 36          | 36          | 93          | 99          | 89          | 85          |
| Place Bread Basket        | 77          | 64          | 81          | 71          | 91          | 93          | 81          | 80          |
| Place Bread Skillet       | 85          | 66          | 77          | 67          | 90          | 93          | 92          | 82          |
| Place Burger Fries        | 94          | 87          | 94          | 94          | 96          | 99          | 95          | 98          |
| Place Can Basket          | 62          | 62          | 49          | 52          | 71          | 69          | 57          | 57          |
| Place Cans Plasticbox     | 94          | 84          | 97          | 98          | 99          | 96          | 37          | 68          |
| Place Container Plate     | 99          | 95          | 97          | 95          | 96          | 100         | 99          | 94          |
| Place Dual Shoes          | 75          | 75          | 79          | 88          | 94          | 88          | 53          | 51          |
| Place Empty Cup           | 100         | 99          | 100         | 98          | 100         | 100         | 91          | 95          |
| Place Fan                 | 87          | 85          | 80          | 75          | 96          | 96          | 77          | 74          |
| Place Mouse Pad           | 60          | 39          | 70          | 70          | 83          | 89          | 58          | 62          |
| Place Object Basket       | 80          | 76          | 44          | 39          | 89          | 88          | 81          | 72          |
| Place Object Scale        | 86          | 80          | 52          | 74          | 90          | 97          | 72          | 74          |
| Place Object Stand        | 91          | 85          | 86          | 88          | 90          | 94          | 78          | 85          |
| Place Phone Stand         | 81          | 81          | 88          | 87          | 97          | 99          | 85          | 88          |
| Place Shoe                | 92          | 93          | 96          | 95          | 96          | 99          | 84          | 87          |
| Press Stapler             | 87          | 83          | 92          | 98          | 90          | 97          | 65          | 76          |
| Put Bottles Dustbin       | 84          | 79          | 74          | 77          | 95          | 90          | 65          | 65          |
| Put Object Cabinet        | 80          | 79          | 46          | 48          | 94          | 89          | 80          | 68          |
| Rotate QRcode             | 89          | 87          | 34          | 33          | 93          | 89          | 72          | 84          |
| Scan Object               | 72          | 65          | 14          | 36          | 89          | 92          | 60          | 52          |
| Shake Bottle Horizontally | 99          | 99          | 100         | 100         | 100         | 100         | 100         | 98          |
| Shake Bottle              | 99          | 97          | 99          | 100         | 100         | 100         | 100         | 99          |
| Stack Blocks Three        | 91          | 76          | 6           | 10          | 95          | 97          | 65          | 67          |
| Stack Blocks Two          | 97          | 100         | 92          | 87          | 100         | 100         | 94          | 91          |
| Stack Bowls Three         | 77          | 71          | 76          | 86          | 80          | 81          | 65          | 72          |
| Stack Bowls Two           | 95          | 96          | 96          | 93          | 92          | 98          | 91          | 97          |
| Stamp Seal                | 79          | 55          | 76          | 82          | 90          | 94          | 60          | 63          |
| Turn Switch               | 62          | 54          | 40          | 61          | 61          | 59          | 33          | 39          |
| <b>Average</b>            | <b>82.7</b> | <b>76.8</b> | <b>72.9</b> | <b>72.8</b> | <b>91.9</b> | <b>91.8</b> | <b>76.4</b> | <b>76.3</b> |

## D Real-World Rollouts

**Task 1: Clear the paper ball into trash bin.**



**Task 2: Stack bowls then place in basket.**



**Task 3: Handoff water bottle into basket.**

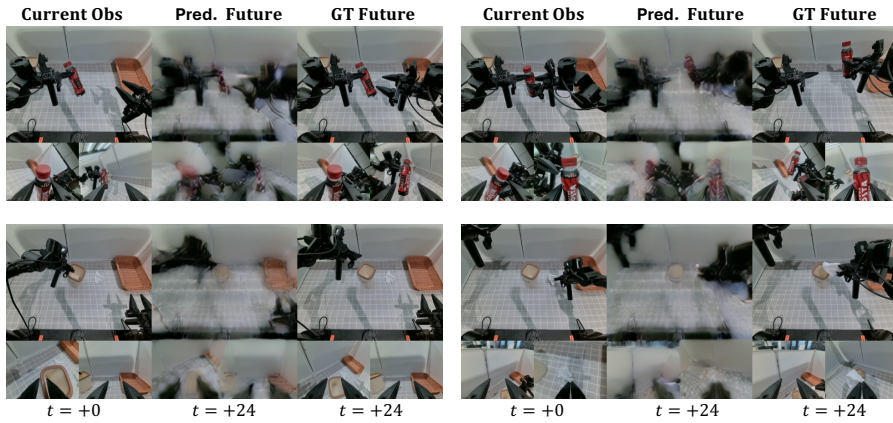


Figure 5: **Additional real-world rollouts.** Rollout frames on three dual-arm tasks and future-video predictions compared with ground-truth future frames.